

Recursion Memoization DP

"Memo జోడించు" – DSA lo ఇది ఒక mantra. కానీ memo **ఒక key మీద** ఆధారపడుతుంది, మరియు ఆ key అసంపూర్ణం అయితే memo **తప్పు జవాబులని పండుతుంది**.

మూడు విరుపులు, మూడూ కొలిచినవి. మొదటిది – `fib(40)` కి 33 కోట్ల calls, memo తో 79. రెండోది – knapsack lo సామర్థ్యాన్ని key నుంచి తీసేస్తే 47.5% తప్పు, మరియు అది ఎక్కువ చెబుతుంది. మూడోది – memoization సమయాన్ని సరిచేస్తుంది, **stack ని కాదు**: $n = 10,000$ దగ్గర RangeError.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే – recursion లోతు పరిమితి కూడా ఈ machine మీద కొలిచినదే.

DSA DEEP 03 యొక్క అదే ప్రశ్న, వేరే వేషంలో

Hashing doc lo: **key** మీ సమానత్వాన్ని సరిగ్గా సూచించాలి. అది నష్టపోతే "99.3% సరైన" జవాబులు వస్తాయి.

DP lo memo అంటే **ఒక hash map**, మరియు దాని key అంటే **స్థితి**. స్థితిలో ఒక కొలత మర్చిపోతే – రెండు వేర్వేరు పరిస్థితులు ఒకే key పంచుకుంటాయి, మరియు మొదటి జవాబు రెండో దానికి **తప్పుగా** ఇవ్వబడుతుంది.

§6 lo కొలుస్తాం: **47.5% తప్పు**, మరియు తప్పు దిశ **ఎక్కువ** – అంటే "నేను 15 విలువ సాధించగలను" అని చెప్పి, నిజంగా 7 మాత్రమే.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. DP అంటే ఏమిటి – మూడు వాక్యాల్లో
2. అడగాల్సిన ప్రశ్నలు – స్థితి ఏమిటి?

Part 2 – మొదటి విరుపు: ఘాతీయ పునరావృత్తి

3. Step – సహజమైన recursion
4. **మొదటి విరుపు** – 33 కోట్ల calls, అందులో 79 ప్రత్యేకమైనవి

Part 3 – రెండో విరుపు: అసంపూర్ణ key

5. Step – memo జోడించడం
6. **రెండో విరుపు** – స్థితిలో ఒక కొలత మర్చిపోతే 47.5% తప్పు
7. Step – స్థితిని ఎలా కనుగొనాలి

Part 4 – మూడో విరుపు: memo stack ని కాపాడడం

8. Step – పైనుంచి కిందికి (top-down)
9. **మూడో విరుపు** – $n = 10,000$ దగ్గర RangeError
10. Step – tabulation · మరియు స్థలం ఆదా చేస్తే ఏమి పోతుంది

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code · 42,000 cases · mutation testing
12. బతికిన మార్పు – ఒక LCS కాదు, చాలా LCS లు
13. Interview lo ఎలా రాయాలి

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. DP అంటే ఏమిటి – మూడు వాక్యాల్లో

1. ఒక పెద్ద సమస్యని చిన్న సమస్యలుగా విడగొట్టడం. 2. ఆ చిన్న సమస్యలు పదే పదే తిరిగి వస్తాయి.
3. కాబట్టి ప్రతి దాన్నీ ఒక్కసారే పరిష్కరించి, జవాబు దాచుకోవడం.

మూడో వాక్యం మాత్రమే DP. మొదటి రెండూ కేవలం recursion.

మరియు "ఒక్కసారే" అనేది ఒక పేరు మీద ఆధారపడుతుంది

"ఈ చిన్న సమస్యని ఇంతకుముందు పరిష్కరించానా?" అని అడగాలంటే, ఆ సమస్యకే ఒక పేరు కావాలి. ఆ పేరే స్థితి (state), మరియు అదే memo యొక్క key.

కాబట్టి DP lo మీరు తీసుకునే ఏకైక ముఖ్యమైన నిర్ణయం: ఈ ఉప-సమస్యని పూర్తిగా వర్ణించే అతి చిన్న సమాచారం ఏమిటి?

"పూర్తిగా" అనే మాట ముఖ్యం. §6 lo దాన్ని కొలుస్తాం.

2. అడగాల్సిన ప్రశ్నలు – స్థితి ఏమిటి?

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఉప-సమస్యని పూర్తిగా వర్ణించే సమాచారం ఏమిటి?	ఇదే memo key – §6
పొడవు/సంఖ్య కావాలా, లేక అసలు జవాబు కావాలా?	§10 – స్థలం ఆదా చేస్తే రెండోది పోతుంది
n ఎంత పెద్దది?	~9,000 దాటితే recursion RangeError (§9)
ఒక్కే వస్తువుని ఎన్నిసార్లు వాడొచ్చు?	ఒకసారే (0/1) vs అపరిమితం – వేర్వేరు loops
జవాబు లేకపోవచ్చా?	-1 vs Infinity – §11 lo d[0]=0 ఎందుకు కీలకమో
స్థితి ఎంత పెద్దది?	states × ఒక్కో state ఖర్చు = మొత్తం సమయం

ఒక ప్రశ్న అడిగితే మరు DP ని నిజంగా అర్థం చేసుకున్నారని తెలుస్తుంది

"ఈ ఉప-సమస్య యొక్క జవాబు, నేను ఇక్కడికి ఎలా వచ్చానో దాని మీద ఆధారపడుతుందా?"

ఆధారపడితే – మీ స్థితి **అసంపూర్ణం**. ఆ "ఎలా వచ్చాను" అనేదాన్ని కూడా స్థితిలో చేర్చాలి.

Knapsack lo: "నేను 3వ వస్తువు దగ్గర ఉన్నాను" అనేది సరిపోదు, ఎందుకంటే జవాబు **ఎంత సామర్థ్యం మిగిలింది** అనేదాని మీద ఆధారపడుతుంది. §6 lo ఆ ఒక్క మరుపు **47.5% తప్పు** జవాబులు ఇస్తుంది.

Part 2 – మొదటి విరుపు: ఘాతీయ పునరావృత్తి

3. Step – సహజమైన recursion

```
const fib = n => n < 2 ? n : fib(n-1) + fib(n-2);
```

ఒక పంక్తి, మరియు **ఖచ్చితంగా సరైనది**. నిర్వచనాన్ని అక్షరాలా అనువదించినది.

4. మొదటి విరుపు – 33 కోట్ల calls, అందులో 79 ప్రత్యేకమైనవి

fib(n) · ఎన్ని function calls, ఎంత సమయం

n	naive calls	naive సమయం	memo calls	memo సమయం
10	177	13 us	19	16 us
20	21,891	502 us	39	4 us
30	26,92,537	6 ms	59	3 us
35	2,98,60,703	51 ms	69	3 us
40	33,11,60,281	571 ms	79	7 us
45	(చాలా ఎక్కువ)	–	89	4 us

FIB(40) → 33,11,60,281 CALLS · MEMO తో 79

నిష్పత్తి **42 లక్షల రెట్లు**.

కారణం సులభమైనది: fib(38) ని naive version **2 కోట్ల సార్లు** లెక్కిస్తుంది, ప్రతిసారి అదే జవాబు.

మరియు అది **సరైన** జవాబు – కేవలం 2 కోట్ల సార్లు.

కానీ మొదటి వరుస చూడండి – N = 10 దగ్గర MEMO నెమ్మది

naive **13 μs**, memo **16 μs**. ఎందుకంటే memo కి ఒక Map తయారు చేయాలి, ప్రతి call కి has / set చేయాలి – మరియు 177 calls కి ఆ ఖర్చు విలువైనది కాదు.

DSA Deep 01 (n ≤ 32 దగ్గర linear వేగం) మరియు Deep 03 (Set, sort కంటే నెమ్మది) – **అదే నమూనా మూడోసారి**.

ప్రతి optimisation ఒక స్థిరాంకాన్ని జేడిస్తుంది. n చిన్నగా ఉన్నప్పుడు స్థిరాంకమే మొత్తం.

Part 3 – రెండో విరుపు: అసంపూర్ణ key

5. Step – memo జోడించడం

fib కి స్థితి స్పష్టం – ఒకే ఒక సంఖ్య n కాబట్టి m.set(n, v) .

ఇప్పుడు ఒక నిజమైన problem: **0/1 knapsack**. బరువులు, విలువలు, ఒక సామర్థ్యం. ప్రతి వస్తువుని తీసుకోవచ్చు లేదా వదలొచ్చు.

```
const go = (i, c) => { // i = ఏ వస్తువు దగ్గర, c = మిగిలిన సామర్థ్యం
  if (i === w.length) return 0;
  let r = go(i+1, c); // వద్దు
  if (w[i] <= c) r = Math.max(r, v[i] + go(i+1, c - w[i])); // తీసుకో
  return r;
};
```

ఇప్పుడు memo జోడించాలి. **Key ఏమిటి?** సహజమైన జవాబు: **i** – ఎందుకంటే అదే మనం ముందుకి కదిలే కొలత.

6. రెండో విరుపు – స్థితిలో ఒక కొలత మర్చిపోతే 47.5% తప్పు

0/1 knapsack · 2,00,000 యాదృచ్ఛిక cases · brute force తో పోల్చినవి

memo key	తప్పు జవాబులు	%
(i, c) – సరైనది	0	0.0%
i మాత్రమే	95,036	47.5%
c మాత్రమే	34,592	17.3%

ఉదా (i మాత్రమే): w=[1,1,3,1,4] v=[5,7,5,3,2] C=1 → 15, నిజం 7
ఉదా (c మాత్రమే): w=[1,4,3,2,4] v=[1,8,5,5,6] C=5 → 11, నిజం 10

జవాబు 15, నిజం 7 – MEMO ఎక్కువ చెప్పింది

C = 1. అంటే బరువు 1 ఉన్న ఒక్క వస్తువు మాత్రమే పట్టుతుంది → గరిష్ట విలువ 7.

కానీ recursion lo go(3, 1) అనే call, మరియు go(3, 0) అనే call – **రెండింటికీ key = 3**. మొదటిది "ఇంకా చోటు ఉంది" అనే పరిస్థితి, రెండోది "చోటు లేదు". Memo మొదటి జవాబుని రెండో దానికి ఇచ్చేస్తుంది.

మరియు తప్పు దిశ ఎక్కువ వైపు. అంటే మే algorithm "నేను 15 సాధించగలను" అని ప్రకటిస్తుంది, మరియు అది సాధించలేని ప్రణాళిక.

మరియు ఇది 52.5% సార్లు సరిగ్గానే పనిచేస్తుంది

C పెద్దగా ఉంటే, లేదా వస్తువులు తక్కువగా ఉంటే – ఒకే i కి రెండు వేర్వేరు c విలువలు ఎప్పుడూ రాకపోవచ్చు. అప్పుడు అసంపూర్ణ key ఎప్పుడూ ఢీకొట్టదు.

LeetCode యొక్క మొదటి రెండు ఉదాహరణలు నెగ్గుతాయి. ఆపై ఒక hidden test విఫలం.

DSA Deep 03 యొక్క "99.3% సరైన key" – అదే వ్యాధి, వేరే problem.

7. Step – స్థితిని ఎలా కనుగొనాలి

ఒక యాంత్రిక పద్ధతి – ఆలోచన అవసరం లేని పద్ధతి

మీ recursive function యొక్క parameters చూడండి. మారే ప్రతి parameter స్థితిలో భాగం.

$go(i, c) \rightarrow$ స్థితి (i, c) . ముగిసింది.

$go(i, j, k) \rightarrow$ స్థితి (i, j, k) .

మారని విషయాలు (w , v arrays) స్థితిలో భాగం కాదు – అవి అన్ని calls కీ ఒకటే.

అంటే key ని "ఆలోచించి" ఎంచుకోవాల్సిన అవసరం లేదు. దాన్ని చదవాలి.

```
const k = i + ',' + c; // రెండు కొలతలూ key lo
if (m.has(k)) return m.get(k);
```

ఆ విభాజకం , కూడా అవసరం – **DSA Deep 03 §11 యొక్క అదే కారణం.** లేకపోతే $(1, 23)$ మరియు $(12, 3)$ రెండూ "123".

స్థితి ఎంత పెద్దదో అదే మీ సమయం

(i, c) అంటే $n \times C$ స్థితులు. ఒక్కో స్థితికి $O(1)$ పని $\rightarrow$ మొత్తం $O(nC)$.

ఇది "polynomial" లా కనిపిస్తుంది, కానీ **C ఒక విలువ, input పరిమాణం కాదు.** $C = 10^9$ అయితే ఇది పనికీరాదు. (దీన్ని pseudo-polynomial అంటారు, మరియు knapsack NP-hard గా ఉండటానికి కారణం అదే.)

కాబట్టి స్థితిని లెక్కించడం రెండు పనులు చేస్తుంది: సరైన key ఇస్తుంది, మరియు సమయాన్ని చెబుతుంది.

Part 4 – మూడో విరుపు: memo stack ని కాపాడడు

8. Step – పైనుంచి కిందికి (top-down)

Memo తో recursion **అందమైనది**: మీరు నిర్వచనాన్ని రాస్తారు, memo వేగం చూసుకుంటుంది. అవసరమైన స్థితులని మాత్రమే లెక్కిస్తుంది.

ఇప్పుడు ఒక పెద్ద n తో ప్రయత్నిద్దాం.

9. మూడో విరుపు – n = 10,000 దగ్గర RangeError

ఈ Node process లో గరిష్ట recursion లోతు: ~9,181

n	memo తో recursion	tabulation
1,000	పనిచేసింది	పనిచేసింది
5,000	పనిచేసింది	పనిచేసింది
10,000	RangeError	పనిచేసింది
50,000	RangeError	పనిచేసింది
2,00,000	RangeError	పనిచేసింది

MEMOIZATION సమయాన్ని సరిచేస్తుంది. **STACK** ని కాదు.

`climbWays(10000)` లో memo ఉంది, కాబట్టి ప్రతి స్థితి ఒక్కసారి లెక్కించబడుతుంది – **సమయం O(n)**.

కానీ మొదటి call `go(10000) → go(9999) → go(9998) → ...` అంటే memo ఒక్కటి నిండకముందే stack మీద **10,000 frames** పేరుకుపోతాయి.

Memo పునరావృత్తిని ఆపుతుంది, లోతుని కాదు.

10. Step – tabulation · మరియు స్థలం ఆదా చేస్తే ఏమి పోతుంది

```
function climbWays(n, MOD = 1000000007) {  
  if (n <= 1) return 1;  
  let a = 1, b = 1;  
  for (let i = 2; i <= n; i++) { const c = (a + b) % MOD; a = b; b = c; }
```

```
return b;
}
```

Stack లోతు **ఒకటి**. n ఎంత పెద్దదైనా.

కానీ tabulation ఒక కొత్త ప్రలోభాన్ని తెస్తుంది: **మొత్తం పట్టిక ఎందుకు ఉంచాలి? రెండు వరుసలు చాలు కదా?**

LCS – పొడవు కావాలా, లేక అసలు string కావాలా?

a = "AGGTABXYZQWERT" · b = "GXTXAYBQWZRT"

విధానం	స్థలం	పొడవు	అసలు string
పూర్తి పట్టిక	0(mn) = 195	8	"GTABQWRT"
రెండు వరుసలు	0(n) = 13	8	నిర్మించలేము ←

m = n = 10,000 అయితే: పూర్తి పట్టిక 0.4 GB · రెండు వరుసలు 78 KB
→ స్థలం 5,001 రెట్లు తగ్గింది, మరియు జవాబు "ఏమిటి" అనేది పోయింది.

రెండు 8 అని చెబుతాయి. ఒకటి మాత్రమే "GTABQWRT" అని చెప్పగలదు

Traceback కి **వెనక్కినదవాలి** – d[i][j] దగ్గర నుంచి d[i-1][j] లేదా d[i][j-1] వైపు. రెండు వరుసలు మాత్రమే ఉంటే ఆ మూడో వరుస **ఇప్పటికే తుడిచేయబడింది**.

5,001 రెట్లు స్థలం ఆదా – మరియు అది ఉచితం కాదు.

కాబట్టి నియమం సులభమైనది

ప్రశ్న "ఎంత" అని అడిగితే → రెండు వరుసలు చాలు. (పొడవు, సంఖ్య, గరిష్ఠం, కనిష్ఠం.)

ప్రశ్న "ఏమిటి" అని అడిగితే → పూర్తి పట్టిక కావాలి. (అసలు subsequence, ఏ నాణేలు, ఏ వస్తువులు.)

మధ్యలో ఒక మార్గం కూడా ఉంది – **Hirschberg's algorithm**: O(n) స్థలంలో string ని తిరిగి నిర్మిస్తుంది, సమయాన్ని రెట్టింపు చేసి. **నేను దీన్ని కొలవలేదు** (§14).

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code • 42,000 cases • mutation testing

తొమ్మిది నియమాలు, అన్నీ brute force తో పోల్చినవి: **knapsack = brute force**, **C పెరిగితే జవాబు తగ్గకూడదు**, **C=0 → 0**, **LCS పొడవు = brute force**, తిరిగి నిర్మించిన string ఆ పొడవే, అది రెండు strings కీ subsequence, **coinChange = నిజం**, **amount 0 → 0**, **climbWays = నేరుగా లెక్కించినది**.

ఆ ఆరో నియమం ముఖ్యమైనది: పొడవు సరిపోవడం సరిపోదు – ఆ string నిజంగా రెండిటిలోనూ ఉందా అని తనిఖీ చేయాలి.

300 runs • 18,000 knapsacks • 12,000 LCS • 12,000 coin problems
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

knaps	18,000
lcs	12,000
coins	12,000
climbs	3,000
impossible	3,771
emptyLcs	4,132

==== mutation testing ====

మార్పు లేని code	→ 0/300 విఫలం
memo key = i మాత్రమే (సామర్థ్యం మర్చిపోతే)	→ 120/120 విఫలం
memo key = c మాత్రమే	→ 120/120 విఫలం
knapsack lo "వద్దు" శాఖ తీసేస్తే	→ 120/120 విఫలం
ఉదా: knapsack(w=[3,3,1],v=[2,3,5],C=1) = 0, ఆశించినది 5	
LCS lo max కి బదులు min	→ 120/120 విఫలం
ఉదా: lcs("baacac","cabc").len = 2, ఆశించినది 3	
LCS traceback lo '>=' ని '>' గా	→ 0/120 విఫలం
LCS traceback lo i-- మరియు j-- మార్చితే	→ 119/120 విఫలం
ఉదా: "bcc" అనేది "cabc" యొక్క subsequence కాదు	
coinChange lo d[0] = 0 తీసేస్తే	→ 120/120 విఫలం
ఉదా: coinChange([8,5,6,6], 25) = -1, ఆశించినది 4	
coinChange lo 'c <= a' ని 'c < a' గా	→ 120/120 విఫలం

COINCHANGE LO D[0] = 0 - ఒక్క పంక్తి, 120/120

"సున్నా మొత్తానికి సున్నా నాణేలు" అనేది స్పష్టంగా అనిపిస్తుంది, కాబట్టి చాలామంది దాన్ని **ఒక formality** గా చూస్తారు.

అది formality కాదు - అది **మొత్తం recursion యొక్క ఆధారం**. అది లేకపోతే ప్రతి $d[a]$ Infinity గా ఉండిపోతుంది, మరియు జవాబు ఎప్పుడూ -1.

DP lo base case ఒక అంచు సందర్భం కాదు - అది ఏకైక నిజం, మిగతాదంతా దాని నుంచే వస్తుంది.

12. బతికిన మార్పు - ఒక LCS కాదు, చాలా LCS లు

ఒక్క మార్పు బతికింది: traceback lo $d[i-1][j] \geq d[i][j-1]$ ని $>$ గా మార్చడం.

3,00,000 cases · LCS traceback: $\geq$ vs $>$

రెండు చెల్లని సార్లు : 0
వేరే string వచ్చిన సార్లు : 44,152 (14.7%)
ఉదా: a="abb" b="bacabc" (పొడవు 2) → "ab" vs "bb"

14.7% సార్లు వేరే STRING · 0 సార్లు తప్పు STRING

"abb" మరియు "bacabc" - రెండింటిలోనూ "ab" ఉంది, "bb" కూడా ఉంది. రెండు పొడవు 2. **రెండు సరైన LCS లు.**

అంటే ఈ పంక్తి సరైనతనాన్ని నిర్ణయించదు - అది **ఏ సమాధానం ఇస్తామో** నిర్ణయిస్తుంది.

DSA Deep 03 lo twoSum యొక్క మొదటి vs చివరి index - **అదే రకం**. ప్రశ్న "ఏదైనా ఒకటి" అని అడిగితే రెండు సరైనవి. ప్రశ్న "అక్షర క్రమంలో మొదటిది" అని అడిగితే - అప్పుడు ఈ పంక్తి భారం మోస్తుంది.

మరియు ఒక ఆచరణాత్మక పరిణామం

మీ పరిష్కారాన్ని ఒక "ఆశించిన output" తో పోల్చే automated judge - **ఈ పంక్తి వల్ల విఫలం కావచ్చు**, మీ జవాబు సరైనదైనా.

అందుకే మంచి problems "ఏ ఒక్క చెల్లుబాటు జవాబైనా" అని చెబుతాయి, మరియు వాటి judge **మీ జవాబుని ధృవీకరిస్తుంది**, పోల్చదు.

నా fuzz కూడా అదే చేస్తుంది - §11 యొక్క ఆరో నియమం "ఇది subsequence అవునా" అని అడుగుతుంది, "ఇదేనా" అని కాదు.

13. Interview lo ఎలా రాయాలి

అడుగు	ఏమి చేయాలి
1	ముందు recursion రాయండి, memo లేకుండా. నిర్వచనాన్ని అనువదించడమే
2	Function యొక్క parameters చదవండి – మారే స్థితి (§7)
3	ఆ parameters అన్నిటినీ key lo పెట్టండి, విభజకంతో
4	states × ఒక్కో state ఖర్చు లెక్కించి బిగ్గరగా చెప్పండి – అదే మీ complexity
5	n పెద్దదైతే (> ~5,000) tabulation కి మార్చండి – §9
6	"ఏమిటి" అని అడిగితే పూర్తి పట్టిక ఉంచండి – §10

ఈ PROBLEM LO మిమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది recursion రాసి, "ఇప్పుడు memo జోడిస్తాను" అని `m.set(i, r)` రాస్తారు – ఎందుకంటే i అనేది loop variable లా కనిపిస్తుంది.

మీరు బదులుగా ఇలా చెప్పండి: "నా function కి రెండు మారే parameters ఉన్నాయి, కాబట్టి స్థితి (i, c) – రెండూ key lo ఉండాలి. c ని వదిలేస్తే, ఒక i దగ్గర వేర్వేరు మిగిలిన సామర్థ్యాలు ఒకే జవాబుని పంచుకుంటాయి. నేను దాన్ని పరీక్షించినప్పుడు అది 47.5% cases lo తప్పు, మరియు ఎప్పుడూ ఎక్కువ వైపు – అంటే సాధించలేని ప్రణాళిక. మరియు స్థితి $n \times C$ కాబట్టి సమయం $O(nC)$, ఇది C ఒక *విలువ* కాబట్టి pseudo-polynomial."

ఇది memo ఎలా రాయాలో తెలుసని కాదు – అది ఎందుకు పనిచేస్తుందో మీకు తెలుసని చూపుతుంది.

14. Follow-ups – bitmask DP, digit DP, స్థితి కుదింపు

"స్థితిలో ఒక సమితి ఉంటుందా?"

"ఏ నగరాలు ఇప్పటికే సందర్శించాను" లాంటిది. $n \leq 20$ అయితే ఆ సమితిని ఒక **bitmask** గా – ఒక పూర్ణ సంఖ్యగా – రాయొచ్చు, మరియు key అవుతుంది (mask, current). స్థితుల సంఖ్య $2^n \times n - n = 20$ కి అది 2 కోట్లు, నిర్వహించదగినది. $n = 30$ కి అది **320 కోట్లు**, కాదు. ఈ హద్దు ఒక సంకేతం: problem lo "n ≤ 20" అని ఉంటే అది దాదాపు ఎప్పుడూ bitmask DP.

"సంఖ్యల మీద DP – '1 నుంచి N వరకు ఎన్ని సంఖ్యల్లో...' అనే problems?"

Digit DP: స్థితి = (ఏ అంకె దగ్గర, ఇప్పటిదాకా హద్దు అంటుకునే ఉన్నామా, ఇంకా ఏ సమాచారం అవసరమో). $N = 10^{18}$ అయినా అంకెలు 19 మాత్రమే, కాబట్టి స్థితులు తక్కువ. నేను దీన్ని కొలవలేదు.

"O(n) స్థలంలో string ని తిరిగి నిర్మించగలమా?"

అవును – **Hirschberg's algorithm**. విభజించి-జయించు: మధ్య వరుసని రెండు దిశల నుంచి లెక్కించి, విభజన బిందువుని కనుగొని, రెండు సగాల మీద పునరావృతం. స్థలం $O(n)$, సమయం రెట్టింపు. నేను దీన్ని

అమలు చేయలేదు, కాబట్టి $\$10$ యొక్క $5,001 \times$ కి ప్రత్యామ్నాయంగా దీనికి నా దగ్గర సంఖ్య లేదు.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **Hirschberg** (ఫైన), **bitmask/digit DP** యొక్క ఆచరణాత్మక హద్దులు, మరియు **recursion లోతు పరిమితిని పెంచడం** (`node --stack-size`) – అది పనిచేస్తుంది కానీ ఎంత వరకు సురక్షితమో నేను పరీక్షించలేదు.

15. ఏమి నేర్చుకున్నాం

1. DP అంటే recursion + ఒక పేరు. ఆ పేరే స్థితి, మరియు అదే memo key.
2. స్థితిని ఆలోచించి ఎంచుకోవద్దు – parameters చదవండి. మారే ప్రతి parameter స్థితిలో భాగం.
3. అసంపూర్ణ key మౌనంగా ఎక్కువ చెబుతుంది. Knapsack lo `i` మాత్రమే → **47.5% తప్పు**, మరియు 15 vs 7.
4. మరియు అది 52.5% సార్లు సరిగ్గానే పనిచేస్తుంది. అందుకే ఇది hidden test దగ్గర బయటపడుతుంది.
5. Memoization సమయాన్ని సరిచేస్తుంది, stack ని కాదు. ఈ machine lo లోతు పరిమితి ~9,181 – $n = 10,000$ దగ్గర RangeError.
6. చిన్న n దగ్గర memo నెమ్మది. `fib(10)` : naive 13 μs , memo 16 μs . మూడో doc lo మూడోసారి అదే నమూనా.
7. "ఎంత" కి రెండు వరుసలు; "ఏమిటి" కి పూర్తి పట్టిక. స్థలం 5,001 రెట్లు తగ్గుతుంది, మరియు జవాబు పోతుంది.
8. Base case ఒక formality కాదు. `d[0] = 0` తీసేస్తే 120/120 విఫలం.
9. కొన్ని పంక్తులు సరైనతనాన్ని కాదు, ఏ జవాబు అనేది నిర్ణయిస్తాయి. LCS traceback: 14.7% వేరే string, 0 తప్పు string.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Memo ఒక cache కాదు – అది ఒక వాగ్దానం.

మీరు ఇస్తున్న వాగ్దానం: "ఈ key ఒకటే అయితే, జవాబు కూడా ఒకటే."

ఆ వాగ్దానం నిజమో కాదో code తనిఖీ చేయదు. అది తప్పయితే crash కాదు, హెచ్చరిక రాదు – memo మీ మొదటి జవాబుని తీసుకుని, అది సరిపోని చోట నమ్మకంగా ఇచ్చేస్తుంది.

కాబట్టి memo రాసే ప్రతిసారి ఒక్క ప్రశ్న: "ఈ key లో లేని ఏదైనా జవాబుని మార్చగలడా?"